



Terahertz Spectroscopy of Materials

Aadil Shamsudheen Mohamed

Indian Institute of Technology, Hyderabad

May 2025

Declaration

I declare that the work presented in this project is my original contribution and has not been accepted elsewhere for any degree or diploma. All sources of information have been acknowledged and cited appropriately. The results and interpretations presented in this document are accurate to my knowledge and belief.

Signature: Aadil Shamsudheen Mohamed

Acknowledgments

I express my sincere gratitude to Dr. Srabani Kar for her guidance and support throughout this project. Her help and insights were crucial to the completion of this work. I am grateful to have a mentor like her who guides me in my journey as a student and helps me become a better researcher with each passing day.

Contents

1	Introduction	7
2	Terahertz Generation	8
2.1	Photoconductive Antenna for THz Generation(PCA)	8
2.2	Optical rectification for THz Generation(OR) . . .	10
3	Terahertz Detection	12
3.1	Electro-Optic Sampling for THz Detection	12
4	Experimental Setup	14
5	Data Analysis	18
5.1	Fourier Transform	18
5.2	Complex Transmission function	19
5.3	Treating the Data and Removing Noise	20
5.4	Complex Refractive Index of the Sample	21
6	Result and Discussion	21

7	Conclusion	22
8	Python Code	23
8.1	Fourier Transform from Time to Frequency domain	23
8.2	Calculation $T(\Omega)$ and Truncation of data	24
8.3	Calculating the Complex Refractive Index of the sample	25
8.4	Plotting the graph for n and k	31

Abstract

This project investigates the complex optical properties of target material samples utilizing Terahertz Time-Domain Spectroscopy (THz-TDS). By capturing time-domain transient electric field waveforms, Fast Fourier Transform (FFT) algorithms were employed to convert time-series data into frequency-domain spectra, enabling the simultaneous determination of both phase shift and amplitude attenuation. From these frequency-dependent transmission parameters, key material properties—including the complex refractive index $\tilde{n}(\omega) = n(\omega) + i\kappa(\omega)$, absorption coefficient $\alpha(\omega)$, and complex dielectric permittivity—were accurately extracted. The results demonstrate clear spectral signatures and resonance profiles characteristic of inter- and intra-molecular modes within the THz regime. This work underscores the efficacy of THz-TDS as a precise tool for non-contact material characterization with broad applications in condensed matter physics, semiconductor analysis, and structural quality control.

1 Introduction

Terahertz radiation falls between infrared and microwave radiation on the electromagnetic spectrum and is often called the “terahertz gap”. Its frequency ranges from 0.1 to 10 terahertz (THz), which has historically been challenging to generate and detect. Recent advancements in sources and detectors have allowed it to be used in diverse fields.

One of its most promising characteristics is its ability to penetrate deeply into materials that are opaque to visible light, like plastics, clothing, and packaging. This unique property is invaluable for non-destructive analysis and quality control in fields like pharmaceuticals and aeronautics. Terahertz imaging can reveal hidden defects, identify foreign objects, and assess the internal structure of materials without causing any damage. In security, THz scanners offer a safe and non-invasive alternative to X-rays for detecting unwanted substances. Unlike X-rays, Terahertz radiation is non-ionizing, posing no risk to human health. There is also research on terahertz radiation in high-speed wireless communication, and it is being investigated for its use in medical fields, such as early cancer detection and monitoring skin conditions.

Terahertz radiation holds immense importance in material science, offering unique capabilities for probing and understanding the fundamental properties of materials. Unlike visible light or X-rays, Terahertz radiation interacts with materials in a distinct way, providing insights into their structure, composition, and dynamics. One of the key applications of Terahertz in material science is in spectroscopy. Many materials exhibit unique spectral fingerprints in the Terahertz range, allowing for the identification and characterization of their constituent molecules, crystal structures, and electronic states. This

technique is invaluable for studying a swath of materials, from semiconductors and polymers to superconductors and 2D materials. This is found by finding how the molecules are placed, which is obtained by Terahertz spectroscopy. [3] Terahertz spectroscopy can also be used to study amorphous materials such as ceramics, which lack a long-range crystallization structure. Terahertz spectroscopy can be used to unravel some of the fundamentals of the materials that lead to their crystallization. At the Terahertz range, the radiation excites the molecules in large parts of the material, allowing weaker forces like Hydrogen bonding and Van der Waals dispersion forces. As a result, it opens a unique perspective to the study of organic molecules.

2 Terahertz Generation

2.1 Photoconductive Antenna for THz Generation(PCA)

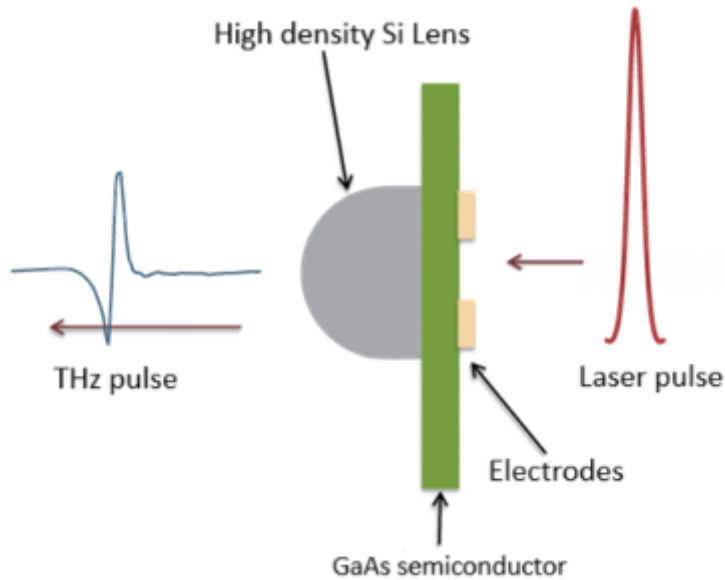


Figure 1: Setup for THz generation using PCA

A photoconductive antenna consists of a semiconductor material with a pair of metal electrodes deposited on its surface. The semiconductor material is typically a material with a short carrier lifetime, such as low-temperature grown Gallium Arsenide (*LT-GaAs*). This allows for rapid generation and recombination of charge carriers, which is essential for generating THz radiation. For THz generation, a femtosecond laser pulse is focused onto the semiconductor material between the metal electrodes. The photons in the laser pulse have enough energy to excite electrons in the semiconductor from the valence band to the conduction band, creating electron-hole pairs. This process temporarily makes the semiconductor conductive. Then a bias voltage is applied across the PCA electrodes. This leads to the electric field created by the bias voltage accelerating the photogenerated electrons and holes in opposite directions, creating a transient current. The accelerating charges constitute a time-varying current that radiates electromagnetic waves. Because the current pulse is very short (on the order of femtoseconds), the emitted electromagnetic waves have a broad frequency spectrum that extends into the THz range. [3]

The transient current density of the semiconductor is given by:

$$J(t) = e \cdot n(t) \cdot v(t)$$

Here e is the charge of an electron, $n(t)$ is time-dependent carrier density generated by the laser pulse and $v(t)$ is the carrier velocity, which depends on the applied bias electric field and material properties. The carrier density $n(t)$ is proportional to the intensity of the laser pulse $I(t)$, which can often be modeled as a Gaussian or secant hyperbolic function.

The electric field emitted by THz, E_{THz} , is proportional to the time derivative of the photocurrent:

$$E_{THz} \propto \frac{dJ(t)}{dt}$$

Expanding the above equation, we get:

$$E_{THz(t)} \propto e \left[\frac{dn(t)}{dt} v(t) + n(t) \frac{dv(t)}{dt} \right]$$

Here, $\frac{dn(t)}{dt}$ represents how quickly carriers are generated and recombined and $\frac{dv(t)}{dt}$ accounts for carrier acceleration under the bias field. The Fourier transform of the THz Electric field gives the output frequency spectrum. The bandwidth of the THz radiation depends on the laser pulse duration(shorter pulse yields broader spectra) and the carrier recombination time in the photoconductive material.

2.2 Optical rectification for THz Generation(OR)

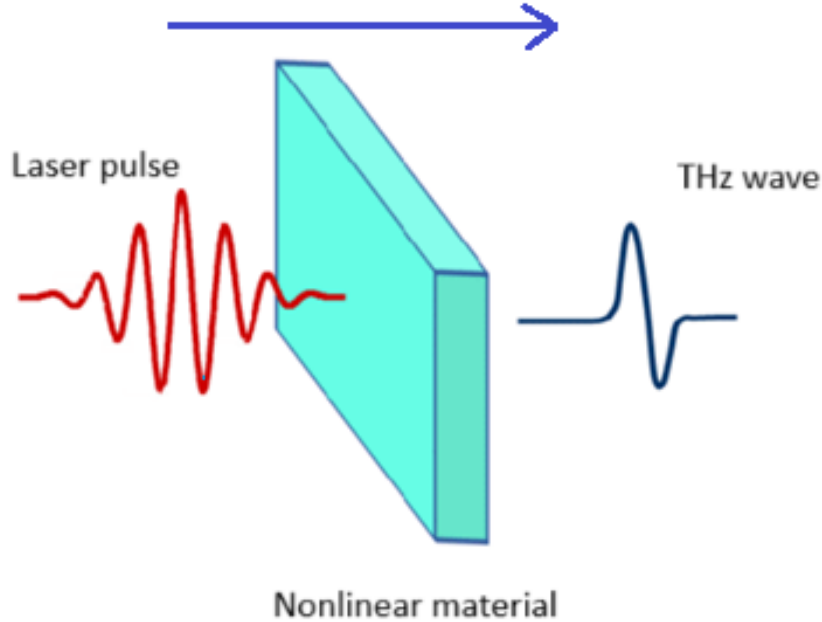


Figure 2: Setup for THz generation using OR

Optical rectification is a non-linear optical process that can be used to generate THz radiation. It involves using an intense laser pulse

to induce a nonlinear polarization in a material, which then radiates THz waves. For the process, a material with a strong second-order nonlinear susceptibility is required, such as a crystal such as Lithium Niobate ($LiNbO_3$) or Gallium Arsenide ($GaAs$). For generation, an intense femtosecond laser pulse is focused onto the nonlinear material. Due to this, the strong electric field of the laser pulse distorts the electron clouds in the material, creating a nonlinear polarization. This polarization oscillates at frequencies that fall within the THz range. The oscillating non-linear polarization acts as a source of THz radiation, emitting electromagnetic waves in the THz frequency range.

Optical rectification arises from the second-order nonlinear polarization $P^{(2)}$, which is proportional to the square of the electric field $E(t)$ of the incident laser pulse:

$$P^{(2)}(t) = \epsilon_0 \chi^{(2)} E^2(t)$$

Here $\chi^{(2)}$ is the second-order nonlinear susceptibility tensor of the material, and ϵ_0 is the permittivity of free space. [5]

The broad spectral bandwidth of femtosecond laser pulses allows mixing of different frequency components within the pulse. This generates a polarization component oscillating at the difference frequencies, which correspond to THz frequencies:

$$f_{THz} = |f_1 - f_2|$$

This process is referred to as optical rectification because it generates low-frequency (THz) components from high-frequency optical fields.

Efficient THz generation requires phase matching between the optical pump pulse and the generated THz wave. This ensures constructive interference and maximizes energy transfer. Phase matching can

be achieved through techniques such as tilted pulse fronts or by using materials with specific birefringence properties.

3 Terahertz Detection

3.1 Electro-Optic Sampling for THz Detection

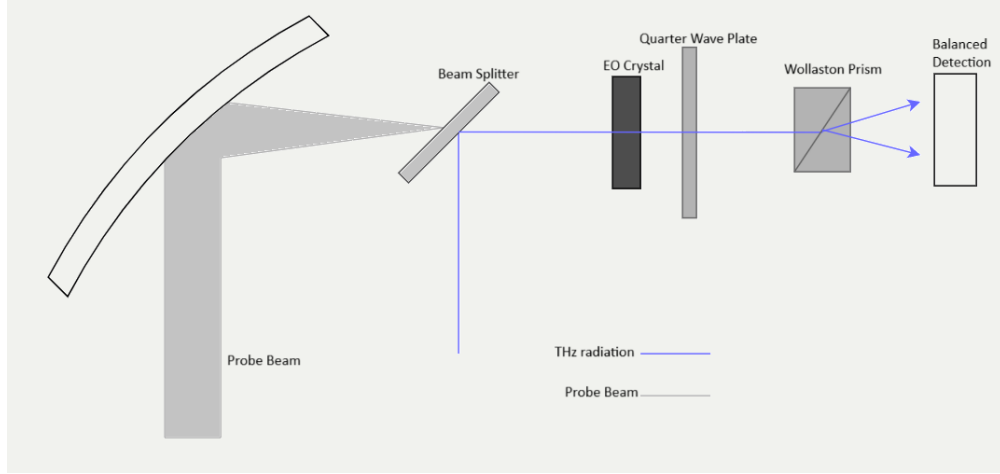


Figure 3: Setup EOS for THz Detection

Electro-optic sampling (EOS) is a widely used technique for detecting terahertz (THz) radiation. It leverages the electro-optic effect, where the refractive index of a crystal changes in response to an applied electric field. Electro-optic sampling provides a highly sensitive and temporally resolved method for measuring the electric field of THz pulses. The THz pulse, whose electric field we want to measure, interacts with an electro-optic crystal. Commonly used crystals include *ZnTe*, *GaP*, and *LiNbO₃*. The THz electric field induces a change in the refractive index of the crystal. This change is proportional to the strength of the THz field.

$$\Delta n = r \cdot E_{THz}$$

Here r is the electro-optic coefficient of the crystal. The above equation is a simplified one and doesn't capture the full complexity, especially when dealing with the tensorial nature of the electro-optic effect. The electro-optic effect is described by a change in the optical indicatrix, which is a mathematical construct representing the refractive index properties of the crystal. The change in the the optical indicatrix (ΔB) is linearly related to the applied electric field. This relationship is expressed using the electro-optic tensor r_{ijk} .

$$\Delta B_{ij} = \sum_k r_{ijk} \cdot E_k$$

Here r_{ijk} are the elements of the electro optic tensor(a third rank tensor) and E_k the component of the electric field in the k direction. The electro-optic tensor r_{ijk} , describes how the electric field in the k direction affects the optical properties related to the i and j directions. This tensor reflects the symmetry properties of the crystal. [1]

The change in refractive index (Δn) is related to the change in the optical indicatrix (ΔB). It depends on the crystal's cut, the polarization of the light, and the direction of the applied electric field. In practical situations, often an effective electro-optic coefficient ('r') is used, which implicitly takes into account the relationship between Δn and ΔB for that particular setup. So here we will use the first equation as it is a simpler relation and gives an approximately good estimate.

A synchronized, short laser pulse (the probe beam) is directed through the electro-optic crystal. This probe beam is polarized. As the probe beam passes through the crystal, the change in refractive index induced by the THz field causes a slight change in the polarization of the probe beam. This change is directly related to the instantaneous strength of the THz electric field at the time the probe pulse passes through the crystal. [4]

$$\Delta\Phi = (2\pi/\lambda) \cdot \Delta n \cdot L$$

Here L is the length of the crystal. The probe beam from the crystal is passed through a quarter wave plate. The quarter-wave plate changes the beam polarization from linear to elliptically polarized. This is done because it can increase the sensitivity of the measurement. The change in polarization of the probe beam is measured using optical components like a Wollaston prism (which splits the beam into two orthogonally polarized components) and a balanced photodetector. The difference in intensity between the two components is proportional to the change in polarization for small changes in polarization.

$$\Delta I \propto \sin(\Delta\Phi) \approx \Delta\Phi$$

Therefore, the change in intensity is approximately proportional to the THz electric field:

$$\Delta I \propto (2\pi/\lambda) \cdot r \cdot E_{THz} \cdot L$$

The balanced photodetector measures the difference in intensity between the two polarization components. This difference is directly proportional to ΔI . By varying the time delay between the THz pulse and the probe pulse, and repeating the measurement, a complete time-domain map of the THz electric field can be reconstructed. This is done by scanning the delay stage, effectively sampling the THz pulse at different points in time. [4]

4 Experimental Setup

A typical Terahertz (THz) generation and detection experimental setup, particularly for THz Time-Domain Spectroscopy (TDS), in-

volves a combination of optical and electronic components. The steps and devices used are briefly discussed below:

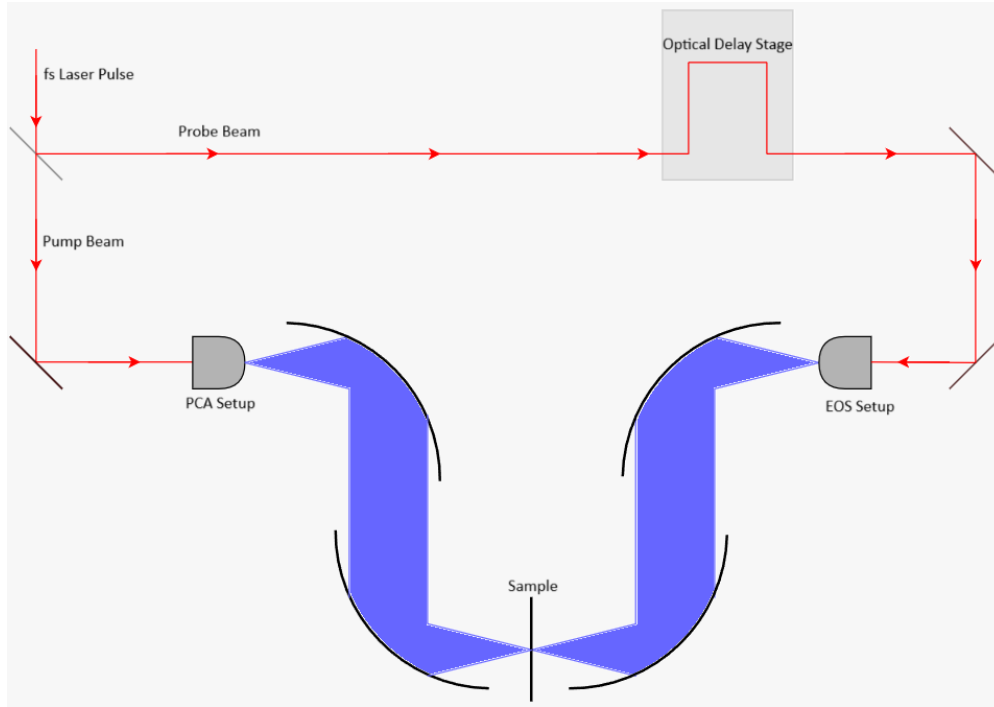


Figure 4: The above image is the diagram of a time-domain spectroscopy setup where a femtosecond laser pulse is passed through a beam splitter from which the pump beam is passed through a PCA setup, the output is passed through mirrors and is focused on the sample from which the pulse is passed through mirrors and focused on the EOS setup where THz is measured. The probe beam is passed through an optical delay stage and then directed to the EOS setup.

- 1) Laser source: A femtosecond laser is used as the laser source. It provides ultrashort pulses of light, typically in the near-infrared range (e.g., around 800nm from a Ti:sapphire laser). This laser beam is then split into two paths, one for THz generation (pump beam) and the other for THz detection (probe beam). [6]
- 2) THz Generation: We have considered two ways to generate THz waves.
 - a) Photoconductive Antenna (PCA): The pump beam is focused onto a PCA(Figure 1), which consists of a semicon-

ductor material with electrodes. The laser pulses generate charge carriers, which, under a bias voltage, produce THz radiation.

- b) Optical Rectification: The pump beam can be focused onto a nonlinear crystal (like *ZnTe* or *LiNbO₃*). This process converts the optical pulses into THz pulses(Figure 2).
- 3) THz Beam Path: Off-axis parabolic mirrors are used to collimate and focus the generated THz beam. These mirrors are preferred because they minimize distortions. The sample under investigation is placed at the focal point of the THz beam. A purge box, utilizing dry air or nitrogen, is often used to minimize the absorption of the THz radiation by water vapor in the air. [6]
- 4) THz Detection using Electro-Optic Sampling (EOS): The probe beam is directed through an electro-optic crystal (e.g., *ZnTe*) exposed to the THz beam(Figure 3). The THz electric field induces a change in the crystal's refractive index, which alters the probe beam's polarization. Polarization optics and a balanced photodetector measure this change, providing a signal proportional to the THz electric field.
- 5) Delay Stage: A precise mechanical delay stage controls the timing of the probe beam relative to the THz beam. By varying the delay, the THz waveform is sampled in the time domain. [6]
- 6) Polarization crystal: A polarizer is used to prepare the probe beam with a specific polarization. After passing through the electro-optic crystal, the polarization of the probe beam is slightly altered due to the presence of the THz field. An analyzer splits the probe beam into two orthogonally polarized components. [6]
- 7) Balanced Photodetector: A balanced photodetector measures

the difference in intensity between the two polarization components of the probe beam. This difference is proportional to the change in polarization induced by the THz field. [6]

- 8) Data Acquisition System: The signal from the balanced photodetector is recorded by a data acquisition system. By scanning the delay stage and recording the signal at each time delay, a time-domain map of the THz electric field is obtained. [6]

The process begins with a femtosecond laser, which emits ultra-short pulses of light. This laser beam is split into two paths: the pump beam and the probe beam. The pump beam is used to generate THz radiation. This is typically done by focusing the pump beam onto a photoconductive antenna or a nonlinear crystal. In the case of a photoconductive antenna, the laser pulses generate charge carriers, which, under a bias voltage, produce THz radiation. With optical rectification, the nonlinear crystal converts the optical pulses into THz pulses. [3]

The generated THz beam is then directed towards the sample under investigation using mirrors. And sent to the EOS setup with mirrors.

For detection, the probe beam is directed through an electro-optic crystal that is exposed to the THz beam. The THz electric field alters the crystal's refractive index, which in turn affects the polarization of the probe beam. By measuring this change in polarization as a function of the time delay between the THz pulse and the probe pulse, the temporal profile of the THz electric field is reconstructed. This is achieved using a balanced photodetector and a delay stage, which precisely controls the timing of the probe beam relative to the THz beam. The recorded signal is then processed by a computer to extract information about the sample's interaction with the THz

radiation. [6]

5 Data Analysis

EOS measures the THz electric field as a function of time delay (τ) between the THz pulse and the probe pulse. This gives a time-domain waveform, $E(t)$, which represents the amplitude of the THz electric field at each point in time.

Depending on the data quality, might need to apply some pre-processing steps:

- 1) Baseline Correction: Remove any DC offset or slow-varying background in the signal.
- 2) Filtering: Apply a digital filter (e.g., a low-pass or band-pass filter) to remove noise or unwanted frequency components.
- 3) Windowing: Apply a window function (e.g., a Hamming or Hanning window) to the time-domain data to reduce spectral leakage in the Fourier transform.

5.1 Fourier Transform

The core of our analysis is to apply the Fourier Transform to the time domain data, $E(t)$. This converts the signal from the time domain to the frequency domain, giving a complex-valued spectrum, $E(f)$. The magnitude of $E(f)$ represents the amplitude of each frequency component in the THz pulse. The phase of $E(f)$ represents the phase relationship between different frequency components.

We do this by applying Fast Fourier Transform(FFT) to the time domain data to get the Fourier Transformed, frequency-based Sample signal amplitude($E_{sam}(f)$) and Reference signal amplitude($E_{ref}(f)$).

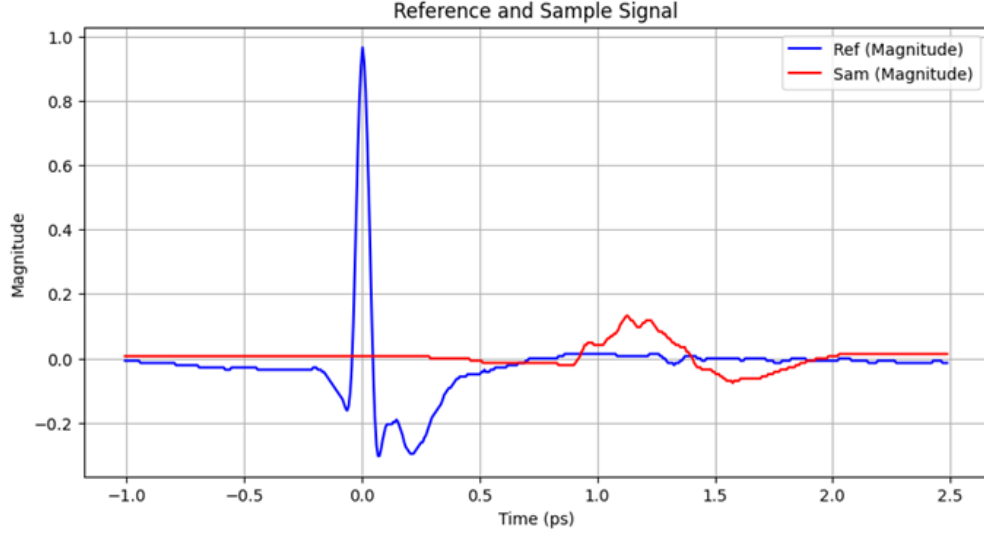


Figure 5: Reference and Sample Data(taken from reference [2])

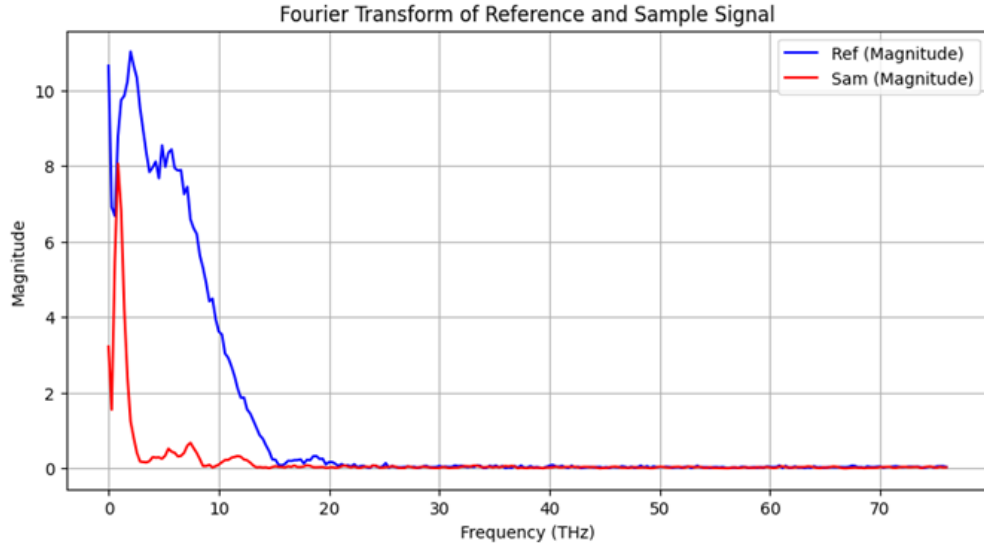


Figure 6: Fourier Transformed Reference and Sample Data

5.2 Complex Transmission function

The complex transmission function($T(\Omega)$) is mathematically it's defined as:

$$T(\Omega) = \frac{E_{sam}(f)}{E_{ref}(f)}$$

Where E_{sam} and E_{ref} are the Fourier transforms of the THz electric field transmitted through the sample and the reference THz electric field, respectively. [2]

The complex transmission function($T(\Omega)$) quantifies how the sample affects the amplitude and phase of the THz pulse as a function of frequency. It includes contributions from:

- Fresnel transmission coefficients at interfaces(air-sample inerfaces)
- Propagation phase through the material
- Fabry–Perot interference effects (multiple internal reflections inside the sample), captured by the $FP(f)$ term. [2]

5.3 Treating the Data and Removing Noise

We truncate the data in the range between 0.98 THz and 15.71 THz. As there the signal reaches near zero from 15.71 THz. So the small noises gets amplified when taking ratios of the E_{sam} and E_{ref} when calculating $T(\Omega)$.

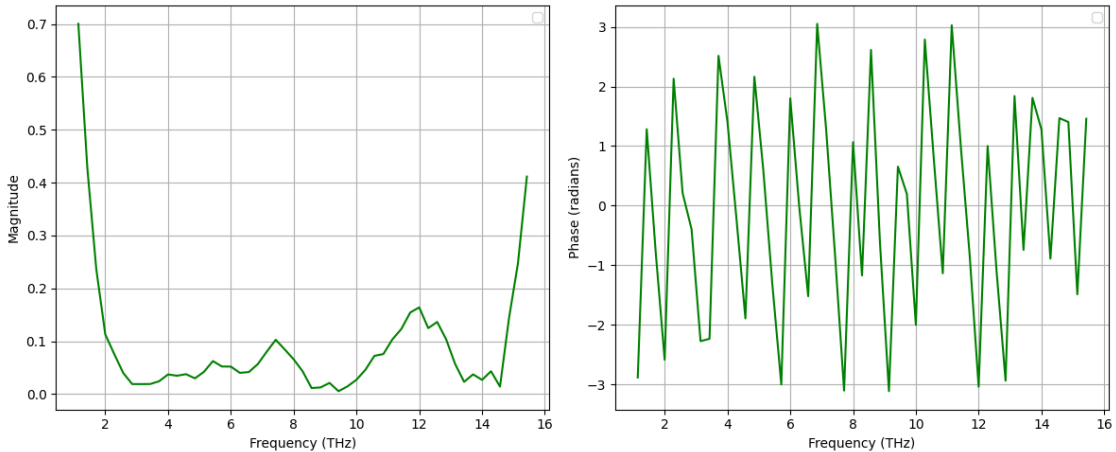


Figure 7: Magnitude and Phase graphs of $T()$ after truncation.

5.4 Complex Refractive Index of the Sample

The complex transmission function $T(f)$ at THz frequencies f , through a sample of thickness d and refractive index $\tilde{n} = n + ik$ can be written as follows:

$$T(\Omega) = \frac{4\tilde{n}_{\text{air}}\tilde{n}}{(\tilde{n}_{\text{air}} + \tilde{n})^2} \cdot \exp \left[-i(\tilde{n} - \tilde{n}_{\text{air}}) \frac{2\pi f d}{c} \right] \cdot FP(f)$$

where n_{air} is the refractive index of dry air ($n_{\text{air}} = 1$) and $FP(f)$ is the Fabry-Perot term. The equation below gives the value for FP, where N is the number of reflections made by the pulse inside the sample: [2]

$$FP(f) = \sum_{m=0}^{N-1} \left\{ \left(\frac{\tilde{n} - \tilde{n}_{\text{air}}}{\tilde{n} + \tilde{n}_{\text{air}}} \right)^2 \cdot \exp \left[-2i\tilde{n} \frac{2\pi f d}{c} \right] \right\}^m.$$

The values of the complex refractive index were found by running the optimization algorithm (code in section 8.3).

6 Result and Discussion

We found the complex refractive index of the 0.47 mm thick sample by using the Complex Transmission Function. Which in turn was calculated using the Fourier transformed sample and reference frequency domain data. The calculation of the complex refractive index was concluded in just a couple of steps, as the data was from a THz Time Domain Spectroscopy. THz-TDS provides both the amplitude and phase in the frequency domain, which leads to less model-dependent complex refractive index determination compared to, Fourier Transform Infrared (FTIR) Spectroscopy. Vector Network Analyzers (VNAs) also give both, but THz-TDS typically offers

a broader bandwidth and simpler setup for many material characterization applications.

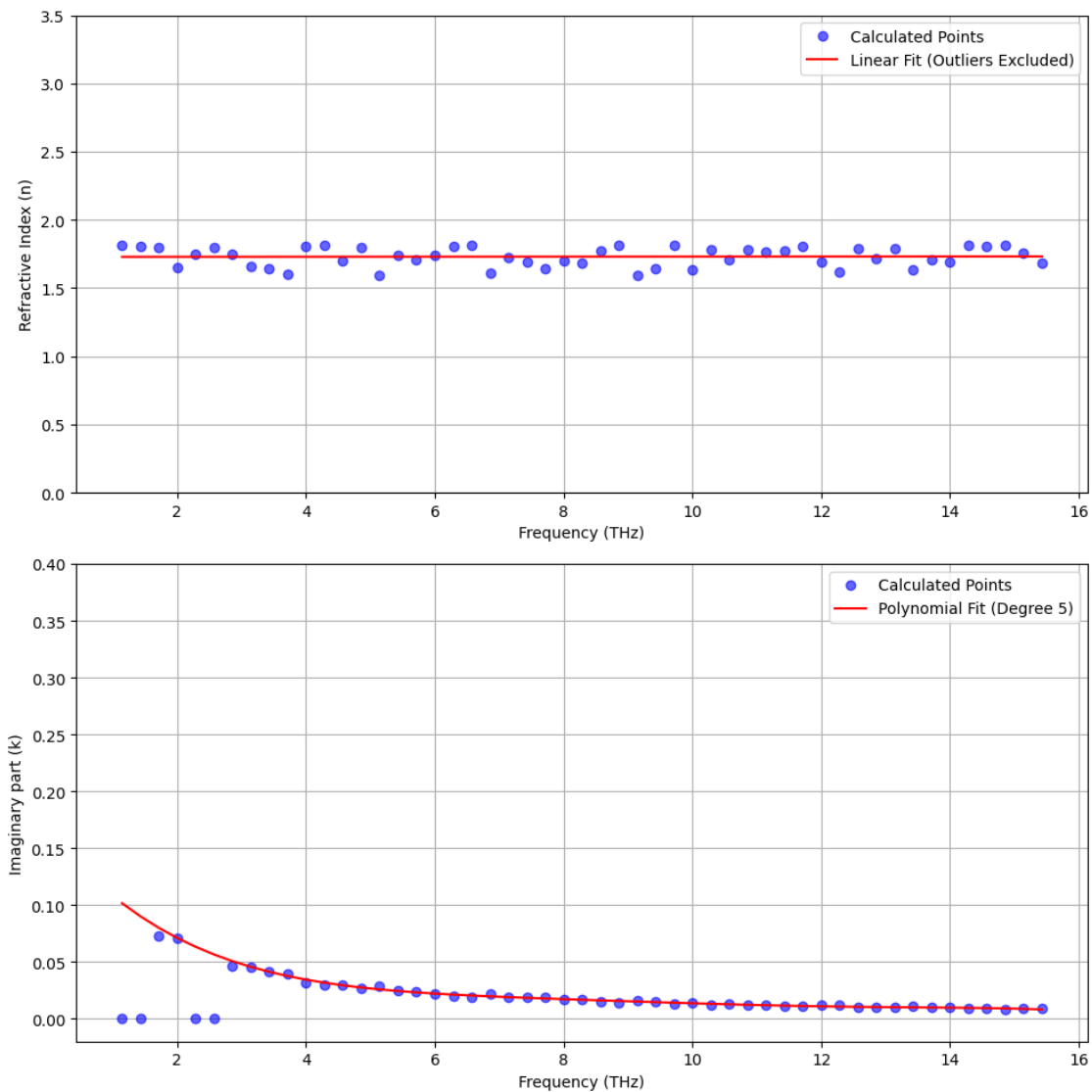


Figure 8: Real and imaginary parts of the complex refractive index converged from the optimization algorithm plotted against the frequency in THz.

7 Conclusion

This project expanded upon the advantages of THz spectroscopy. Expanded on the topic of Terahertz generation and Terahertz detection. Later explained the experimental setup of a Terahertz Time

Domain Spectroscopy using a PCA for THz generation and an EOS for THz detection.

This project successfully demonstrated the advantage of using a THz-TDS data of a sample to find the complex refractive index of the sample. Similarly, THz-TDS can also be used to find other properties of materials, demonstrating THz-TDS's potential in material science.

8 Python Code

8.1 Fourier Transform from Time to Frequency domain

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 file_path = "data1.txt" #Time domain data
6
7 data = pd.read_csv(file_path,delim_whitespace=True, comment="%",
8                     names=["time", "Ref", "Sam"])
9
10 time = data["time"].values
11 ref_signal = data["Ref"].values
12 sam_signal = data["Sam"].values
13
14 dt = np.mean(np.diff(time))
15 fs = 1 / dt
16
17 freqs = np.fft.fftfreq(len(time), d=dt)
18 fft_ref = np.fft.fft(ref_signal)
19 fft_sam = np.fft.fft(sam_signal)
20
21 freq1 = len(freq)//2
```

```

22 # Plot for FFT'ed signal
23 plt.figure(figsize=(10, 5))
24 plt.plot(freqs[:freq1], np.abs(fft_ref[:freq1]), label="Ref (Magnitude)",
           color='b')
25 plt.plot(freqs[:freq1], np.abs(fft_sam[:freq1]), label="Sam (Magnitude)",
           color='r')
26 plt.xlabel("Frequency (THz)")
27 plt.ylabel("Magnitude")
28 plt.title("Fourier Transform of Reference and Sample Signal")
29 plt.legend()
30 plt.grid()
31 plt.show()
32
33 # Plot for raw signal
34 plt.figure(figsize=(10, 5))
35 plt.plot(time, ref_signal, label="Ref (Magnitude)", color='b')
36 plt.plot(time, sam_signal, label="Sam (Magnitude)", color='r')
37 plt.xlabel("Time (ps)")
38 plt.ylabel("Magnitude")
39 plt.title("Reference and Sample Signal")
40 plt.legend()
41 plt.grid()
42 plt.show()
43
44 print("Sampling Frequency (fs):", fs, "Hz")

```

8.2 Calculation $T(\Omega)$ and Truncation of data

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 Tomega = fft_sam / fft_ref
5
6 print(f"Original data points: {len(freqs)}")
7 valid_range_mask = (freqs >= 0.98) & (freqs <= 15.71)

```

```

8 freqs = freqs[valid_range_mask]
9 Tomega = Tomega[valid_range_mask]
10 print(f"Truncated data points (0-20 THz): {len(freqs)}")
11
12 freqs_hz = freqs * 1e12
13
14 # Plot of Tomega magnitude and phase
15 plt.figure(figsize=(12, 5))
16
17 plt.subplot(1, 2, 1)
18 plt.plot(freqs, np.abs(Tomega), color='green', linewidth=1.5)
19 plt.xlabel("Frequency (THz)")
20 plt.ylabel("Magnitude")
21 plt.legend()
22 plt.grid(True)
23
24 plt.subplot(1, 2, 2)
25 plt.plot(freqs, np.angle(Tomega), color='green', linewidth=1.5)
26 plt.xlabel("Frequency (THz)")
27 plt.ylabel("Phase (radians)")
28 plt.legend()
29 plt.grid(True)
30
31 plt.tight_layout()
32 plt.show()

```

8.3 Calculating the Complex Refractive Index of the sample

```

1 # Physical constants and parameters
2 n_air = 1
3 d = 0.47e-3 # m
4 c = 299792458 # m/s
5 N = 3

```

```

6
7 def preprocess_phase_data(phase_data):
8     """Remove 2 discontinuities from phase data"""
9     return np.unwrap(phase_data)
10
11 def calculate_frequency_weights(freqs, snr_data=None,
12     target_range=(1.587, 1.817)):
13     if snr_data is not None:
14         weights = snr_data / np.max(snr_data)
15     else:
16         weights = np.ones_like(freqs)
17         good_freq_mask = ((freqs >= 5.0) & (freqs <= 15.0))
18         weights[good_freq_mask] *= 2.2
19         low_freq_mask = (freqs < 4.0)
20         weights[low_freq_mask] *= 0.2
21         mid_freq_mask = (freqs >= 4.0) & (freqs < 5.0)
22         weights[mid_freq_mask] *= 0.6
23     return weights
24
25 def smart_initial_guess_generator(freq_hz, target_n_range=(1.587, 1.817)):
26     n_center = np.mean(target_n_range)
27     freq_thz = freq_hz / 1e12
28     if freq_thz < 2.0:
29         n_guess = n_center + 0.015
30     elif freq_thz > 12.0:
31         n_guess = n_center - 0.010
32     else:
33         n_guess = n_center
34     n_guess = np.clip(n_guess, target_n_range[0] + 0.005,
35         target_n_range[1] - 0.005)
36     k_guess = 0.012
37     return [n_guess, k_guess]
38
39 def FP_f(n_tilde, f, N):
40     r_sq = ((n_tilde - n_air) / (n_tilde + n_air)) ** 2
41     x = r_sq * np.exp(-2j * n_tilde * (2 * np.pi * f * d / c))

```

```

40     if np.abs(1 - x) < 1e-12:
41         return float(N)
42     return (1 - x**N) / (1 - x)
43
44 def error_function(n_k_pair, Tomega_measured, f, N, weight=1.0):
45     n_tilde = complex(n_k_pair[0], n_k_pair[1])
46     try:
47         calculated_Tomega = (
48             (4 * n_air * n_tilde / (n_air + n_tilde) ** 2)
49             * np.exp(-1j * (n_tilde - n_air) * (2 * np.pi * f * d / c))
50             * FP_f(n_tilde, f, N)
51         )
52         residual = (Tomega_measured - calculated_Tomega) * weight
53         return np.array([residual.real, residual.imag])
54     except (OverflowError, ZeroDivisionError, RuntimeWarning):
55         return np.array([1e6, 1e6])
56
57 def global_error_function(n_k_pair, Tomega_measured, f_hz, N, weight=1.0):
58     residuals = error_function(n_k_pair, Tomega_measured, f_hz, N, weight)
59     return residuals[0]**2 + residuals[1]**2
60
61 def assess_convergence_quality(result, n_result, k_result,
62                                target_range=(1.587, 1.817)):
63     if hasattr(result, 'fun'):
64         residual_norm = result.fun if np.isscalar(result.fun) else
65         np.sum(result.fun**2)
66     else:
67         residual_norm = result
68     n_reasonable = 1.5 <= n_result <= 1.9
69     k_reasonable = 0.0 <= k_result <= 0.4
70     in_target_range = target_range[0] <= n_result <= target_range[1]
71     high_quality = residual_norm < 0.03
72     excellent_quality = residual_norm < 0.0005
73     score = 0
74     if n_reasonable: score += 1
75     if k_reasonable: score += 1

```

```

74     if in_target_range: score += 3
75     if high_quality: score += 1
76     return {
77         'residual_norm': residual_norm,
78         'physically_reasonable': n_reasonable and k_reasonable,
79         'in_target_range': in_target_range,
80         'high_quality': high_quality,
81         'excellent_quality': excellent_quality,
82         'overall_score': score,
83         'priority_score': score + (3 if excellent_quality else 0)
84     }
85
86 def global_optimization(Tomega, f_hz, N, weight, target_range=(1.587,
1.817)):
87     hard_bounds = [(target_range[0], target_range[1]), (0.0, 0.25)]
88     best_result = None
89     best_quality = {'priority_score': -1, 'residual_norm': float('inf')}
90     try:
91         de_result = differential_evolution(
92             global_error_function,
93             bounds=hard_bounds,
94             args=(Tomega, f_hz, N, weight),
95             seed=42,
96             maxiter=500,
97             atol=1e-9,
98             tol=1e-9,
99             popsize=25,
100            strategy='best1bin',
101            workers=1
102        )
103        if de_result.success:
104            quality = assess_convergence_quality(de_result.fun,
105            de_result.x[0], de_result.x[1], target_range)
106            if quality['priority_score'] > best_quality['priority_score']:
107                best_result = de_result
108                best_quality = quality

```

```

108     except Exception:
109         pass
110     for attempt in range(3):
111         try:
112             initial_guess = smart_initial_guess_generator(f_hz,
113 target_range)
114             bh_result = basinhopping(
115                 global_error_function,
116                 initial_guess,
117                 minimizer_kwargs={
118                     'method': 'L-BFGS-B',
119                     'bounds': hard_bounds,
120                     'args': (Tomega, f_hz, N, weight)
121                 },
122                 niter=200,
123                 T=0.03,
124                 stepsize=0.02,
125                 seed=42 + attempt
126             )
127             if bh_result.lowest_optimization_result.success:
128                 quality = assess_convergence_quality(
129                     bh_result.fun, bh_result.x[0], bh_result.x[1],
130 target_range
131                 )
132                 if quality['priority_score'] >
133 best_quality['priority_score']:
134                     best_result = bh_result
135                     best_quality = quality
136             except Exception:
137                 continue
138         try:
139             n_starts = 18
140             n_range = np.linspace(target_range[0] + 0.002, target_range[1] -
141 0.002, n_starts)
142             k_range = np.linspace(0.005, 0.035, 10)
143             for n_start in n_range:

```

```

140         for k_start in k_range:
141             try:
142                 local_result = minimize(
143                     global_error_function,
144                     [n_start, k_start],
145                     args=(Tomega, f_hz, N, weight),
146                     bounds=hard_bounds,
147                     method='L-BFGS-B',
148                     options={'ftol': 1e-12, 'gtol': 1e-12}
149                 )
150                 if local_result.success:
151                     quality = assess_convergence_quality(
152                         local_result.fun, local_result.x[0],
153                         local_result.x[1], target_range
154                     )
155                     if quality['priority_score'] >
156                     best_quality['priority_score']:
157                         best_result = local_result
158                         best_quality = quality
159                     except Exception:
160                         continue
161             except Exception:
162                 pass
163             return best_result, best_quality
164
165 def calculate_complex_refractive_index(freqs_thz, freqs_hz, Tomega, N):
166     n_complex_values = []
167     converged = []
168     quality_metrics = []
169     target_range = (1.587, 1.817)
170     freq_weights = calculate_frequency_weights(freqs_thz,
171     target_range=target_range)
172     if hasattr(Tomega, 'imag') and np.any(np.imag(Tomega) != 0):
173         phase_data = np.angle(Tomega)
174         unwrapped_phase = preprocess_phase_data(phase_data)
175         magnitude_data = np.abs(Tomega)

```

```

173     Tomega_processed = magnitude_data * np.exp(1j * unwrapped_phase)
174 else:
175     Tomega_processed = Tomega
176     for i, (f_thz, f_hz) in enumerate(zip(freqs_thz, freqs_hz)):
177         current_weight = freq_weights[i]
178         result, quality = global_optimization(
179             Tomega_processed[i], f_hz, N, current_weight, target_range
180         )
181         if result is not None and quality['physically_reasonable']:
182             if hasattr(result, 'x'):
183                 n_result, k_result = result.x
184             else:
185                 n_result, k_result = result.lowest_optimization_result.x
186             n_complex_values.append(complex(n_result, k_result))
187             converged.append(True)
188             quality_metrics.append(quality)
189         else:
190             n_complex_values.append(complex(np.nan, np.nan))
191             converged.append(False)
192             quality_metrics.append({'residual_norm': float('inf'),
193 'priority_score': 0})
194     return np.array(n_complex_values), np.array(converged),
195         quality_metrics
196
197
198 n_complex, conv_flags, quality_info =
199     calculate_complex_refractive_index(freqs, freqs_hz, Tomega, N)

```

8.4 Plotting the graph for n and k

```

1 # Outlier Detection and Removal for n_real (using IQR) ---
2 n_real_converged = np.real(n_complex[conv_flags])
3 freqs_converged = freqs[conv_flags]

```

```

4
5 Q1_n = np.percentile(n_real_converged, 25)
6 Q3_n = np.percentile(n_real_converged, 75)
7 IQR_n = Q3_n - Q1_n
8
9 # Define outlier bounds for n
10 lower_bound_n = Q1_n - 1.5 * IQR_n
11 upper_bound_n = Q3_n + 1.5 * IQR_n
12
13 # Create a mask for non-outliers for n
14 non_outlier_mask_n = (n_real_converged >= lower_bound_n) &
    (n_real_converged <= upper_bound_n)
15
16 # Filter the data for n
17 n_real_filtered = n_real_converged[non_outlier_mask_n]
18 freqs_filtered_for_n = freqs_converged[non_outlier_mask_n]
19
20 print(f"Original converged points for n: {len(n_real_converged)}")
21 print(f"Points identified as n-outliers: {len(n_real_converged) -
    len(n_real_filtered)}")
22 print(f"Points remaining for n-fit: {len(n_real_filtered)}")
23
24
25 # Outlier Detection and Removal for k_imag (using Residuals from Initial
    Fit + IQR)
26 k_imag_converged = np.imag(n_complex[conv_flags]) # Ensure using
    n_complex from previous cell
27
28 # 1. Fit a 5th-degree polynomial to all converged k points initially
29 if len(k_imag_converged) > 5:
30     k_fit_initial = np.polyfit(freqs_converged, k_imag_converged, 5)
31     k_curve_initial = np.poly1d(k_fit_initial)
32
33 # 2. Calculate residuals
34 residuals_k = k_imag_converged - k_curve_initial(freqs_converged)
35

```

```

36     # 3. Use IQR on residuals to identify outliers
37     Q1_res_k = np.percentile(residuals_k, 25)
38     Q3_res_k = np.percentile(residuals_k, 75)
39     IQR_res_k = Q3_res_k - Q1_res_k
40
41     # Define outlier bounds for residuals
42     lower_bound_res_k = Q1_res_k - 1.5 * IQR_res_k
43     upper_bound_res_k = Q3_res_k + 1.5 * IQR_res_k
44
45     # Create a mask for non-outliers based on residuals
46     non_outlier_mask_k = (residuals_k >= lower_bound_res_k) &
(residuals_k <= upper_bound_res_k)
47
48     # Filter the data for k based on residual outliers
49     k_imag_filtered = k_imag_converged[non_outlier_mask_k]
50     freqs_filtered_for_k = freqs_converged[non_outlier_mask_k]
51
52     print(f"\nOriginal converged points for k: {len(k_imag_converged)}")
53     print(f"Points identified as k-outliers (based on residuals):
{len(k_imag_converged) - len(k_imag_filtered)}")
54     print(f"Points remaining for k-fit: {len(k_imag_filtered)}")
55
56     # 4. Refit the 5th-degree polynomial to the filtered k data
57     if len(k_imag_filtered) > 5:
58         k_fit = np.polyfit(freqs_filtered_for_k, k_imag_filtered, 5)
59         k_curve = np.poly1d(k_fit)
60         print(f"Polynomial fit coefficients for k (degree 5, after
residual outlier removal): {k_fit}")
61     else:
62         k_curve = lambda x: np.nan
63         print("Not enough non-outlier points (after residual analysis) to
fit a 5th degree polynomial to k.")
64
65 else:
66     k_curve = lambda x: np.nan
67     non_outlier_mask_k = np.zeros_like(k_imag_converged, dtype=bool)

```

```

68     freqs_filtered_for_k = np.array([])
69     k_imag_filtered = np.array([])
70     print("\nNot enough converged points to perform initial 5th degree
polynomial fit for k outlier detection.") # Updated print
71
72
73 # Fitting for n (remains the same, using IQR on raw n data)
74 if len(n_real_filtered) > 1:
75     n_fit = np.polyfit(freqs_filtered_for_n, n_real_filtered, 1)
76     n_line = np.poly1d(n_fit)
77     print(f"\nLinear fit coefficients for n (after outlier removal):
{n_fit}")
78 else:
79     n_line = lambda x: np.nan
80     print("\nNot enough non-outlier points to fit a line to n.")
81
82
83 # Plotting
84 plt.figure(figsize=(10, 10))
85
86 plt.subplot(2, 1, 1)
87 # Plot all original calculated points for n
88 plt.plot(freqs[conv_flags], n_real[conv_flags], 'o', label='Calculated
Points', alpha=0.6, color='blue')
89 # Plot linear fit line for n (using the fit from filtered data)
90 plt.plot(freqs_converged, n_line(freqs_converged), '-', color='red',
label='Linear Fit', linewidth=1.5)
91
92 plt.xlabel("Frequency (THz)")
93 plt.ylabel("Refractive Index (n)")
94 plt.grid(True)
95 plt.ylim(0.0, 3.5)
96 plt.legend()
97
98 plt.subplot(2, 1, 2)
99 # Plot all original calculated points for k

```

```

100 plt.plot(freqs[conv_flags], k_imag[conv_flags], 'o', label='Calculated
    Points', alpha=0.6, color='blue')
101 # Plot polynomial fit line for k (using the fit from filtered data)
102 plt.plot(freqs_converged, k_curve(freqs_converged), '-', color='red',
    label='Polynomial Fit (Degree 5)', linewidth=1.5)
103
104 plt.xlabel("Frequency (THz)")
105 plt.ylabel("Imaginary part (k)")
106 plt.grid(True)
107 plt.ylim(-0.02, 0.4)
108 plt.legend()
109
110 plt.tight_layout()
111 plt.show()

```

References

- [1] H. J. Bakker, G. C. Cho, H. Kurz, Q. Wu, and X.-C. Zhang. Distortion of terahertz pulses in electro-optic sampling. *J. Opt. Soc. Am. B*, 15(6):1795–1801, Jun 1998.
- [2] Francesco D’Angelo, Zoltán Mics, Mischa Bonn, and Dmitry Turchinovich. Ultra-broadband thz time-domain spectroscopy of common polymers using thz air photonics. *Optics Express*, 22(10):12475–12485, 2014.
- [3] Jens Neu and Charles A. Schmuttenmaer. Tutorial: An introduction to terahertz time domain spectroscopy (thz-tds). *Journal of*

- Applied Physics*, 124(23):231101, 12 2018.
- [4] Paul C. M. Planken, Han-Kwang Nienhuys, Huib J. Bakker, and Tom Wenckebach. Measurement and calculation of the orientation dependence of terahertz pulse detection in znte. *J. Opt. Soc. Am. B*, 18(3):313–317, Mar 2001.
- [5] György Tóth, Gyula Polónyi, and János Hebling. Tilted pulse front pumping techniques for efficient terahertz pulse generation. *Light: Science & Applications*, 12:256, 2023.
- [6] Jiayu Zhao. Principle of terahertz time-domain spectroscopy. *arXiv*, 2023.